

Dalek Invaders

Senior Project

Brian Cowan

11/22/2013

Table of Contents

Introduction.....	5
Game Play.....	5
Research.....	7
Buffer Strategy.....	7
Canvas.....	7
Graphics.....	8
Walkthrough.....	9
Game.....	9
JFrame Display.....	9
Buffer Strategy.....	10
Initializing Entities.....	10
Game Loop.....	11
Firing a shot.....	12
Moving the Daleks faster.....	12
Playing Sound.....	13
Key Press Events.....	13
Entity.....	14
Load Entity.....	14
Move Entity.....	15

TARDIS Entity	15
Moving the TARDIS.....	15
TARDIS collisions.....	15
Dalek Entity	16
Updating the Game	16
Do Logic	16
Loading Images.....	17
Challenges.....	19
Window Size	19
Keyboard Input	19
Movement	20
Collision Detection	20
Strings and Integers.....	21
Music and Sound Effects	21
Boss Battles.....	22
Buffer Strategy	22
Future Ideas.....	24
New Enemies	24
Regeneration	24
New Levels	25

Works Cited	26
Dalek Invaders Code.....	27
Game	27
Image Loader	39
Sprite	41
Entity	42
TARDIS Entity	44
Shot Entity	45
Dalek Entity	46
Dalek Battle Ship Boss Entity.....	47
PowerPoint Presentation	48
Title	49
Overview	50
Purpose.....	51
Programming Environment.....	52
Timeline	53
What is next?.....	56
Challenges	57
Questions.....	59
Screen Shots.....	60

Introduction

Dalek Invaders is a retro styled game based off of the popular British Broadcasting Company television show, Doctor Who. Doctor Who began broadcasting on November 23, 1963 and has become one of the most popular shows in the world in the past fifty years. Famous for its cheesy lines, old fashioned 1970s style graphics and special effects, and a mysterious alien who travels the universe through space and time, making a Space Invaders type of game and meshing it with Doctor Who seemed to make perfect sense.

Dalek Invaders was created using Java and implementing JFrame, JPanel, and Canvas. The JFrame and JPanel are used for our gaming screen. They displays the game for the user to play and allow us to create a menu for more control over the game. The Canvas and Buffer Strategy was the most difficult part of creating this game. The Canvas makes it possible for us to display our images onto the JFrame and implementing it using a Buffer Strategy makes it so the images do not flicker while they move, creating smoother looking game play. This will be looked into with greater detail within Research and Challenges.

Game Play

The goal of the game is to prevent the Dalek invasion from taking over the Earth. The Daleks are the Doctor's most terrifying enemy. Ever since the beginning they have been there trying to take over planets and kill the Doctor. Because the Doctor is so fond of Earth, specifically London, the Daleks have targeted the planet as their next invasion. If they reach the Earth, all of humanity will either be turned into Daleks themselves or exterminated. In Dalek Invaders, every level more Daleks will come to invade, each wave faster than the last. After every 5 levels, you face the Dalek Battle Ship. If you can destroy the ship, you are rewarded with

a faster firing rate so as to keep up with the ever increasing speed of the invading Daleks.

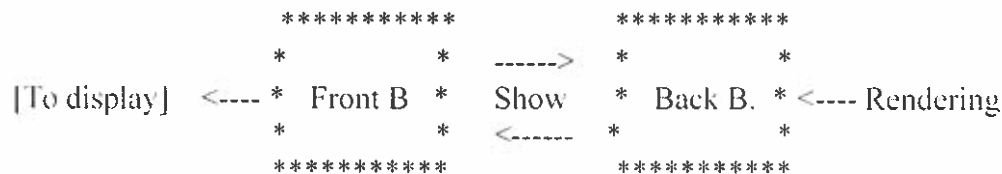
Anyone who watches Doctor Who will know that the Daleks have always and will always be attacking the Doctor and where ever he may go. This is why the game can go on and on. The Doctor can never fully beat the Daleks and every time he beats them one time, they come back stronger than ever. The best strategy to beat each level is not to destroy on line at a time, but to move around as much as you can and destroy the Daleks closest to reaching the Earth, then follow up with destroying a few behind and around him so you can focus on the other side of the battlefield some. If you just focus on one side or one line at a time, you will be quickly overwhelmed.

When it is time to face the boss, you will see the massive ship appear in the place of the Daleks. The Gamer must hit the boss ship thirty-five times the first time he faces it. After that, if he manages to face the ship again, he must hit it thirty-five more time plus an extra multiple of six times. So the second time he faces the boss, it must be shot 41 times. The next time 47 times and so on. As the game goes on, you get upgrades to keep up with the Daleks, but they will continue to come.

Research

Buffer Strategy

Many hours of research was done to complete Dalek Invaders. The majority of the research coming from looking up how to implement buffer strategies with the canvas. What the buffer strategy does is render the images off screen so that when they are displayed to the gamer, there is no flickering and the game can be played smoothly.



The Image is rendered in the background, then moved to the front to be displayed to the gamer. There are a few different ways to do this. Dalek Invaders uses this double buffer. If you use a triple buffer and third level, Mid B, goes between Front and Back. This is used for higher end graphics, but because Doctor Who is never about graphics but only emotion, the game shoots for the same thing.

Canvas

To use the Buffer Strategy, you must implement the java.awt.Canvas component. Because I wanted to use the buffer strategy to smooth the images, Canvas must be incorporated. The canvas lays across the JFrame like a painter laying a canvas over an easel. The easel is the painters sturdy back to hold the canvas so he can create his work of art. The JFrame container

acts the exact same way. The container holds the game while the canvas displays it in all of its glory.

Graphics

So now we have a canvas to create on and a buffer strategy to make it look nice and neat. The next bit of research to be done involves the graphics. The `java.awt.Graphics` class allows us to paint our buffered images onto our canvas laying on the `JFrame`. This class gave us the capability to paint the canvas black like the night sky, display our images in their correct X and Y coordinates, and write all of our text to the screen. It was coming to this step that a problem was discovered. It is not possible to simply write an integer to the screen with the `Graphics`. Also, because the canvas is laying across the `JFrame`, you cannot simply print the text. After some research and figuring this out, the only way to display the score and level to the canvas was to convert them to Strings and then display those instead.

```
g.drawString("Score: ", 0, 10);
    //Converts the score to a String to display to the Gamer
    Score = Integer.toString(score);
    g.drawString(Score, 40, 10);
    g.drawString("Level: ", 100, 10);
    //Converts the level to a String to display to the Gamer
    Level = Integer.toString(level);
    g.drawString(Level, 140, 10);
```

It is here in the `gameLoop()` that we do this. every time the game loops around it takes the current score and converts it into a String to display. It then does the same for the level.

Walkthrough

Game

JFrame Display

```
//Create the container Frame for our game and give it a title
JFrame container = new JFrame("Dalek Invaders");
JPanel panel = (JPanel) container.getContentPane();
//Sets the dimensions of our game
panel.setPreferredSize(new Dimension(790, 610));
setBounds(0, 0, 790, 610);
panel.add(this);
container.pack();
//Makes it so the Gamer cannot resize the game screen
container.setResizable(false);
//Display our game
container.setVisible(true);
//When the screen is closed, close the game and everything running in it
container.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
//Display the game at the center of the computer screen
container.setLocationRelativeTo(null);
```

It is here that we create our container and make it visible to the gamer. The first line will create the container and give it the title of our game, Dalek Invaders. The second line uses a JPanel. This allows us to manipulate the JFrame Container. The third line will set the dimensions of the game while the seventh line will make set resizable to false so the user cannot manually alter the game screen. The fourth through sixth lines will set the bounds the entities in the game will have to be contained in. Before doing anything further, you must make the container visible. This is what the eighth line does. The following line makes it so if the gamer closes the container, the frame exits as well. Without this line the game would continue to play even though it is not on the screen. The last line will display the container at the center of the screen. Such a thing may be overlooked normally, but without this line the container would be displayed at the top left side of the screen where you would have to drag it closer to the more normal position near the center. This line saves the gamer of that trouble.

Buffer Strategy

```
//Creates a basic 2 buffer strategy to load and display the images
createBufferStrategy(2);
strategy = getBufferStrategy();
//Prepare for war
initEntities();
```

These three lines make everything run smooth. The first line creates our Buffer Strategy. With a strategy of two, the game can now load the images off screen and then display them with no flickering. Without this the game would be virtually unplayable due to the vast movement within this game causing flickering.

Initializing Entities

```
tardis = new TardisEntity(this, "tardis.png", 370, 530);
//Display the TARDIS
entities.add(tardis);
//If you have reached a level of multiple 6,
//it is time for a boss battle
if ((level % 6) == 0) {
    for (int i = 0; i < bossLife; i++) {
        //Get the Dalek battle ship
        Entity boss = new BossEntity(this, "final.jpg", 250, 50);
        //Display the ship
        entities.add(boss);
    }
} else {
    dalekCount = 0;
    for (int row = 0; row < 5; row++) {
        for (int x = 0; x < 12; x++) {
            //Get the Daleks
            Entity dalek = new DalekEntity(this, "dalek.png", 100 + (x * 50),
(50) + row * 50);
            //Display the 60 Daleks
            entities.add(dalek);
            //Increase the count of Daleks invading
            dalekCount++;
        }
    }
}
```

To initialize the entities, the game creates and displays the TARDIS at the center of the screen. It then looks at what level it is. Based on this it either adds the Boss to the screen or adds 60 Daleks to the screen.

```

if ((level % 6) != 0) {
    for (int i = 0; i < entities.size(); i++) {
        Entity entity = (Entity) entities.get(i);
        entity.setHorizontalMovement(entity.getHorizontalMovement() * (1 +
level / 10));
    }
}
//If you beat the boss, you are rewarded with more shots in less time
if ((level % 6) == 0) {
    firingInterval = firingInterval - 50;
}

```

It is in Start Game that we decide how difficult or easy to make the game. If it is a level without a boss, we make the Daleks drop faster and faster each round. If it is a Boss level, however, we increase the firing rate of the TARDIS and let the gamer have a little bit easier time for five more levels.

Game Loop

```

while (gameRunning) {
    //Set delta to the current time - the last time the loop ran
    long delta = System.currentTimeMillis() - lastLoopTime;
    //Reset the last loop time to the current time
    lastLoopTime = System.currentTimeMillis();
    //Draw our graphics using our Buffer Strategy
    Graphics2D g = (Graphics2D) strategy.getDrawGraphics();
    g.setColor(Color.black);
    //Fill the background with a black color
    g.fillRect(0, 0, 800, 600);
    //Make the text white
    g.setColor(Color.white);
    g.drawString("Score: ", 0, 10);
    //Converts the score to a String to display to the Gamer
    Score = Integer.toString(score);
    g.drawString(Score, 40, 10);
    g.drawString("Level: ", 100, 10);
    //Converts the level to a String to display to the Gamer
    Level = Integer.toString(level);
    g.drawString(Level, 140, 10);
}

```

It is in the game loop where we begin the heart of the game. Here we calculate delta for moving the Daleks and determining if a shot can be fired or not. We then use our strategy we

made earlier on line three to draw the graphics smoothly with no flickering. After this we display the background and text onto the screen.

Firing a shot

```
//Check to see if enough time has passed since the last fired shot
if (System.currentTimeMillis() - lastFire < firingInterval) {
    //If not enough time has passed, return and do not shoot
    return;
}

//If enough time has passed, set the last fired shot to the current time
lastFire = System.currentTimeMillis();
//Get the shot image and start it's trajectory from the top of the TARDIS
ShotEntity shot = new ShotEntity(this, "shot.gif", tardis.getX() + 10,
tardis.getY() - 20);
//Display the shot
entities.add(shot);
//If the Gamer wants sound effects on
if(soundEffect)
    //Play the shot sound effect
    playShot();
```

When trying to fire a shot, you must first check that enough time has passed since the last shot was fired. If it has then display the shot onto the screen right above the TARDIS and start moving upward toward the Daleks.

Moving the Daleks faster

```
for (int i = 0; i < entities.size(); i++) {
    Entity entity = (Entity) entities.get(i);

    if (entity instanceof DalekEntity) {
        //Increase the horizontal speed of the Daleks by 2%
        entity.setHorizontalMovement(entity.getHorizontalMovement() * 1.02);
    }
}
```

To make each level harder as the level is being played, as the level goes on, the Daleks begin to move faster toward the bottom of the screen, giving the gamer a sense of urgency.

Playing Sound

```
public void playAudio() {
    try {
        //Get the background music file
        in = new FileInputStream(new File("./I_am_the_Doctor.wav"));
        //Convert the file into an AudioStream
        as = new AudioStream(in);
        //Play the background music
        AudioPlayer.player.start(as);
        //Inform the game that the music is playing
        songPlaying = true;
    } catch (FileNotFoundException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    } catch (IOException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
}
```

To play any of the audio files, first the game goes and retrieves the .wav file. It then takes that file and converts it into an Audio Stream. After that it starts playing the music and informs the game that music is playing.

Key Press Events

```
public void keyPressed(KeyEvent e) {
    //Alerts the game that a key has been pressed
    if (getKey() {
        return;
    }
    //If the <- arrow key is pressed, move left
    if (e.getKeyCode() == KeyEvent.VK_LEFT) {
        leftPressed = true;
    }
    //If the -> arrow key is pressed, move right
    if (e.getKeyCode() == KeyEvent.VK_RIGHT) {
        rightPressed = true;
    }
    //If the space bar is pressed, fire
    if (e.getKeyCode() == KeyEvent.VK_SPACE) {
        firePressed = true;
    }
}

public void keyReleased(KeyEvent e) {
    //Alerts the game that a key has been released
    if (getKey() {
```

```

        return;
    }
    //If the <- arrow key is released, stop moving
    if (e.getKeyCode() == KeyEvent.VK_LEFT) {
        leftPressed = false;
    }
    //If the -> arrow key is released, stop moving
    if (e.getKeyCode() == KeyEvent.VK_RIGHT) {
        rightPressed = false;
    }
    //If the space bar is released, stop firing
    if (e.getKeyCode() == KeyEvent.VK_SPACE) {
        firePressed = false;
    }
}

```

To get keyboard input, the game listens for certain key events. If it is any random key and the game is waiting for a key to be pressed, `if (getKey)` will return and let the game know a key has been pressed. Otherwise it waits for the left or right arrow keys or the space bar to be pressed. If one of them is pressed, it alerts the game that something is pressed and to move the TARDIS or fire the shot accordingly. Once the gamer releases the key, the game is notified to stop moving or stop firing.

Entity

Load Entity

```

public Entity(String ref, int x, int y) {
    this.sprite = LoadImages.get().getSprite(ref);
    this.x = x;
    this.y = y;
}

```

This will load the sprite of the image from the Sprite class. It will also retrieve the x and y coordinates for the image.

Move Entity

```
public void move(long delta) {  
    x += (delta * dx) / 1000;  
    y += (delta * dy) / 1000;  
}
```

In order to move the entity, we retrieve delta from game and use it to move the entity across the screen by multiplying by the horizontal and vertical movement and dividing by 1000.

TARDIS Entity

Moving the TARDIS

```
public void move(long delta) {  
    //If we have reached the edge of the screen on the left, do not move  
    if((dx < 0) && (x < 10)) {  
        return;  
    }  
    //If we have reached the edge of the screen on the right, do not move  
    if((dx > 0) && (x > 750)) {  
        return;  
    }  
    super.move(delta);  
}
```

In order to move the TARDIS we must first check that we are not against one of the borders. If we are, we should not move at all. If we are not, then we can inform it that it can move at the speed of delta.

TARDIS collisions

```
public void collidedWith(Entity other) {  
    //If the TARDIS has hit a Dalek, stop the music and end the game  
    if(other instanceof DalekEntity) {  
        game.stopAudio();  
        game.playDeath();  
        game.notifyDeath();  
    }  
    //If the TARDIS has hit the boss, stop the music and end the game  
    if(other instanceof BossEntity) {  
        game.stopAudio();  
        game.playDeath();  
        game.notifyDeath();  
    }  
}
```

```
}  
}
```

If collision detection has occurred and it informs the TARDIS and a Dalek or the Boss, we must stop the music and notify the game that we have lost.

Dalek Entity

Updating the Game

```
if ((dx < 0) && (x < 10)) {  
    game.updateGame();  
}  
    //If the Daleks reach the end of the screen on the right,  
    //drop down the screen and reverse direction  
if ((dx > 0) && (x > 750)) {  
    game.updateGame();  
}
```

If the Daleks hit one of the borders we need to inform the game to do something special. In this case we call doLogic

Do Logic

```
public void doLogic() {  
    //Reverse direction  
    dx = -dx;  
    //Drop down on the screen  
    y += 10;  
    //If the Daleks reach the bottom of the Frame and have not yet touched the  
TARDIS,  
    //notify the game that the Daleks have invaded  
    if (y > 540) {  
        game.notifyDeath();  
    }  
}
```

This will reverse their direction so they travel away from the border to the other side of the screen and will also drop their Y coordinate so they fall toward the Earth. If they reach the bottom of the screen the game is notified of your death.

Loading Images

In order to buffer our images before displaying them, they must be loaded into the game and created into Sprites.

```
private HashMap<String, Sprite> images = new HashMap<String, Sprite>();

//Get the images from the cache
public Sprite getSprite(String ref) {
    //If the image is already cached, use that image
    if (images.get(ref) != null) {
        return (Sprite) images.get(ref);
    }

    //If the image is not in the cache, go get the image from Class Loader
    BufferedImage sourceImage = null;
```

The HashMap is really fast and therefore can receive the images quickly. We then have getSprite which gets each image from the HashMap. If it is not in the HashMap, we have to get them from the Class Loader

```
try {
    //Retrieves the image resource
    URL url = this.getClass().getClassLoader().getResource(ref);
    //If the image cannot be found, print out the error
    if (url == null) {
        System.err.println("Can't find ref: " + ref);
        System.exit(0);
    }

    //Use ImageIO to read the image in
    sourceImage = ImageIO.read(url);
} catch (IOException e) {
    System.err.println("Failed to load: " + ref);
    System.exit(0);
}
```

Here we try to get the image from the class loader. If it does not exist we inform the game of the error. Once found, we read it into our ImageIO.

```
//Create the image
GraphicsConfiguration gc =
GraphicsEnvironment.getLocalGraphicsEnvironment().getDefaultScreenDevice().getDefault
Configuration();
```

```
Image image = gc.createCompatibleImage(sourceImage.getWidth(),
sourceImage.getHeight(), Transparency.BITMASK);

//Draw the image
image.getGraphics().drawImage(sourceImage, 0, 0, null);

//Cache the image for future use to avoid buffer issues
Sprite sprite = new Sprite(image);
images.put(ref, sprite);
```

We then use the Graphics to create the image sprite and draw it to our Canvas. After this we want to save the image for future use to speed up the loader, so we cache it into our HashMap images for easy access. Because of all of this images are displayed extremely fast with no flickering after any movement. This makes the whole game run fast and smooth for an enjoyable experience.

Challenges

Some major challenges needed to be overcome to complete Dalek Invaders. These challenges range from things that needed a little bit of research to things that needed many weeks of research and attempts to get right.

Window Size

Determining the correct window size to display the game sounds unimportant, but actually takes some thought, research, and some testing to find the perfect size. If your game screen is too small it will anger the gamers and they will not want to play the game any longer. On the other hand, if the game screen is made too large it may not fit onto every computer screen the right way, say a small notebook laptop. Not only this, but the smaller the screen, the faster the Daleks will reach each border and drop closer to you. You could lower the amount of Daleks in the game, but this also lowers the enjoyment factor. Make the screen too large and you are left with Daleks that take forever to drop toward the Earth, lowering the excitement, or you add more Daleks and it becomes too overwhelming on the eyes. Dalek Invaders was ultimately built on a 800x600 screen size as it is one of the more common screen sizes and allows for exciting and challenging game play.

Keyboard Input

In Data Structures we learned how to handle keyboard input from the user. This gave me some knowledge in the subject, however that was in C++, not java. As we have been told many times, the two languages are far from alike in many aspects. Using the knowledge from building Gac-Man in Data Structures I was able to determine what was needed for moving the TARDIS across the screen. With further research that C++ code was then able to be converted into Java

code to be used. Research was also needed for action handlers for the menu bar options and how to handle when the gamer selects an option.

Movement

Finding the correct speeds for every entity took some time. Much research and many hours were put in to finding a way to use the time to control the entities. The Daleks move to each tick of time as does the TARDIS when the user requests it to move. The shot also moves to the time, but uses it for another very important reason. If the gamer was just able to hold the space bar down and fire a barrage of shots at the Daleks, it would not be fun. The shot uses the time to determine when the last shot went off. If the gamer wants to fire again, it checks the current time and determines if enough time has passed since the last shot. If it has, a shot is allowed to fire. If not, nothing happens and the gamer has to wait a little longer. To make the game more challenging to the gamer, as time passes, the Daleks begin to speed up drastically, making them even more threatening. When you beat a boss your firing rate increases by decreasing the amount of time the gamer must wait before firing again. This gives the game a great challenge and after a few levels the gamer is rewarded for his efforts.

Collision Detection

Collision Detection was the next big challenge. There were many things that could collide at any time and to keep track of everything proved to be difficult. Daleks could hit the border on the left or right or even the bottom. They could also be hit by the shot fired by the TARDIS. The TARDIS needed to be aware of the left and right borders also, as well as the Daleks running into it. In addition, after all of these collisions something needs to happen and what happens depends on what entity collides with what. If the Daleks collide into one of the left or right borders, they need to drop closer to the Earth and reverse direction. If they collide into

the bottom of the screen or the TARDIS, the game needs to stop and alert the gamer of game over. If the TARDIS hits the left or right border, however, it should not reverse direction but instead stop moving until the gamer reverses it's direction. The shot needs to check if it has hit a Dalek and if it has it needs to add itself and the Dalek to the items to be removed list. If it misses a Dalek and runs off the top of the screen it needs to add itself to the items to be removed list.

Strings and Integers

To display text with the Graphics feature on the frame, you need to use `g.drawString()`. Within this you need to have the message to draw and where on the screen you would like the message to appear. After some research and finding this line this was not difficult. However this presented a problem. One downside to using these graphics is there is no way to draw an integer to the screen. The most efficient way to fix this problem was to convert the integers of Level and Score into Strings and then use `g.drawString` to draw these to the canvas.

Music and Sound Effects

Music should not have been difficult as we have done this before. However this project gave me great trouble when trying to play background music. After attempting six different ways of playing music and all of them failing, I discovered that the .wav file being used was corrupted. After changing the file to a different song everything worked great. Taking it to new heights. Dalek Invaders allows the gamer to determine if he wants music or sound effects to play or be muted. These are given as separate options as in many games, gamers may not care for the music but still want to hear the sound effects or, vice versa, the sound effects may be annoying and they just want to listen to the music. Because of this the gamer has an option of leaving one on while continuing to play the other via the menu bar option Music.

Boss Battles

To add a little variation to the game, after every 5 levels you face the Dalek Battle ship. Creating a new class was not hard as it follows a similar pattern as the Daleks themselves. One of the big problems that arose was that after shooting it just once you would win immediately. That doesn't make for much of a boss fight. To fix this, thirty-five or more ships are printed at the same time in the same spot and move at the same rate. That makes it so the gamer must shoot the boss thirty-five or more times before it reaches the Earth.

Buffer Strategy

The hardest challenge faced while creating Dalek Invaders came early on and took over a month of research to figure out. After that it changed the whole way the game was created and built, taking up even more research and time. The problem being faced was the flickering of images as they moved. Dalek Invaders always has something going on and something is always moving somewhere on the screen, so flickering of images would destroy the entire game and leave it unplayable. In every class where we made some sort of game with moving parts, from Visual Basic all the way up to Data Structures, this was a problem. The dropping black circles in the Visual Basic final project flickered. The race in Java had images that raced across the screen that flickered. Gac-Man had many parts that flickered every time you moved. This was unacceptable at this level and it was absolutely necessary to be fixed. After much research a component called Buffer Strategy was found. The Buffer Strategy class represents the mechanism with which to organize complex memory on Java Canvas. Hardware and software limitations cause the flickering of images but depending on your hardware and software, the images can be given a buffer so that they flow cleanly across the canvas. After the frame has been created, you create the Buffer Strategy. Within this you can decide on how many buffers

you want to use to render the images. The most common number chosen is two, which is what Dalek Invaders uses. Any more than three is useless and one is sometimes not enough. Because the creation of the strategy causes an `IllegalStateException`, the canvas needs to be added to a `Container`, which you see in the code. To use the Buffer Strategy you use `DrawGraphics`. Because we are using two buffers, the graphics are drawn off screen before anything is displayed. This completely eliminates flickering and allows the buffer to clear without the gamer noticing any disturbance in the game play. The buffer then redraws everything and since buffering is done off screen, there is no flickering while waiting for the images. This is essential for the animation because you need to redraw the entire screen as all of the entities within Dalek Invaders move and collide. Once the buffer strategy was implemented and canvas was researched and being used properly, Dalek Invaders started to come together.

Future Ideas

New Enemies

There are big plans for Dalek Invaders in the future. First off, the Daleks are not the only enemy of the Doctor. Some of the more popular ones are the Cybermen, the Silence, and possibly the scariest of them all, the Weeping Angels. Sequels or extra secret levels are planned to be made with these enemies invading Earth. They would come with their own sprites and sound effects.

Regeneration

One of the most genius things about Doctor Who that has kept it on the air for 50 years is the fact that the Doctor is constantly changing. In the past 50 years there have been 11 different men who have played the Doctor. It is an ability he possesses that allows him to change a dying body into a new restored body. It is a process called "regeneration". This amazing ability would be wonderful to have included in Dalek Invaders. Every Doctor is different and every TARDIS is different as well. Dalek Invaders will honor that and when you start the game, you will start off in a black and white TARDIS with the original black and white Daleks. You will face the Daleks as long as you can and when you eventually die, instead of Game Over, you will regenerate. The TARDIS will become the second Doctor's TARDIS, the Daleks will be upgraded, and the game will continue all the way up until the eleventh Doctor and his new and improved TARDIS with the infamous red, white, and blue Daleks invading. This will allow the Gamer to feel more like he really is the Doctor travelling through time and will bring a certain charm to the game that will remind him even more of the show. At the moment these sprites do not exist, but when they are made this will be the very next feature added into Dalek Invaders.

New Levels

One of my favorite games growing up was Frogger for the Play Station. The first level was the classic Frogger game every one knows and loves. When you beat that level, you were then sent to new unimaginable places for the time. It was still the classic Frogger, but this time you would be running down the road instead of across it. These extra levels were amazing as a kid and putting that kind of extra spark into Dalek Invaders would be important. With this classic level of watching your enemy fall from the sky and stopping them before they reach the ground, other levels could include Daleks that have secretly invaded the Earth already and having to stop them in a type of side scrolling adventure.

Works Cited

"BufferStrategy (Java Platform SE 7)." BufferStrategy (Java Platform SE 7). N.p., n.d. Web. 22 Nov. 2013.

"Canvas (Java Platform SE 7)." Canvas (Java Platform SE 7). N.p., n.d. Web. 22 Nov. 2013.

"Game Programming Snippets." *How to Use BufferStrategy in Java*. N.p., 14 Oct. 2010. Web. 22 Nov. 2013.

"JFrame (Java Platform SE 7)." JFrame (Java Platform SE 7). N.p., n.d. Web. 22 Nov. 2013.

"JOptionPane (Java Platform SE 7)." JOptionPane (Java Platform SE 7). N.p., n.d. Web. 22 Nov. 2013.

Dalek Invaders Code

Game

```
//A Canvas is a blank rectangular area of the screen
//onto which the application can draw images and
//Get input from the Gamer
import java.awt.Canvas;
//Frame that will hold our game.
//Supports JFC/Swing component architectures
import javax.swing.JFrame;
//Used to create the menu and menu items
import javax.swing.JMenu;
import javax.swing.JMenuBar;
import javax.swing.JMenuItem;
//For our pop up boxes
import javax.swing.JOptionPane;
import javax.swing.JPanel;
//Used for the graphics and colors throughout the game
import java.awt.Color;
import java.awt.Graphics2D;
//Sets the dimensions for our game
import java.awt.Dimension;
//Action handlers to get input and react
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.KeyAdapter;
import java.awt.event.KeyEvent;
//Buffer Strategy to display our images smoothly
//onto the Canvas
import java.awt.image.BufferStrategy;
//Array Lists to hold our entities
import java.util.ArrayList;
//Used for the music and sound effects
import sun.audio.*;
import java.io.*;

public class Game extends Canvas {

    /**
     *
     */
    private static final long serialVersionUID = -2615286622582969273L;
    //Array List of all of the entities in the game
    private ArrayList<Entity> entities = new ArrayList<Entity>();
    //Array List with destroyed entities (Daleks or shot)
    private ArrayList<Entity> removeList = new ArrayList<Entity>();
    //Used for the loop that runs as the game is played
    private boolean gameRunning = true;
    //Alerts the game to get a keyboard input
    private boolean getKey = true;
    //Move the TARDIS left
    private boolean leftPressed = false;
```

```

//Move the TARDIS right
private boolean rightPressed = false;
//Fire a shot at the Daleks
private boolean firePressed = false;
//Update the game if necessary
private boolean update = false;
//Lets the game know if music is playing or not
private boolean songPlaying = false;
//Lets the game know if the Gamer wants sound effects or not
private boolean soundEffect = true;
//String representation of the level
private String Level;
//String representation of the score
private String Score;
//Default message when the game begins
private String message = "The Daleks are invading!";
//Our TARDIS (Time And Relative Dimension In Space)
private Entity tardis;
private double moveSpeed = 300;
//Time since the last shot was fired
private long lastFire = 0;
//Interval of time that a shot can be fired
private long firingInterval = 500;
//Number of Daleks left invading
private int dalekCount;
//Life of the boss left
private int bossLife = 35;
//Your score
private int score = 0;
//What level you are on
private int level = 1;
//BufferStrategy to buffer images
private BufferStrategy strategy;
//Used to retrieve and play music and sound effects
InputStream in;
AudioStream as;

public Game() {
    //Create the container Frame for our game and give it a title
    JFrame container = new JFrame("Dalek Invaders");
    JPanel panel = (JPanel) container.getContentPane();
    //Sets the dimensions of our game
    panel.setPreferredSize(new Dimension(790, 610));
    setBounds(0, 0, 790, 610);
    panel.add(this);
    container.pack();
    //Makes it so the Gamer cannot resize the game screen
    container.setResizable(false);
    //Display our game
    container.setVisible(true);
    //When the screen is closed, close the game and everything running in it
    container.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    //Display the game at the center of the computer screen
    container.setLocationRelativeTo(null);
    //Create a menu bar

```

```

JMenuBar mb = new JMenuBar();
//Add File to the menu bar
JMenu file = new JMenu("File");
mb.add(file);
//Add Music to the menu bar
JMenu music = new JMenu("Music");
mb.add(music);
//Add Help to the menu bar
JMenu help = new JMenu("Help");
mb.add(help);
//Add Restart as an option under File
JMenuItem restart = new JMenuItem("Restart");
file.add(restart);
//Add Exit as an option under File
JMenuItem exit = new JMenuItem("Exit");
file.add(exit);
//Add Play Music as an option under Music
JMenuItem play = new JMenuItem("Play Music");
music.add(play);
//Add Stop Music as an option under Music
JMenuItem stop = new JMenuItem("Stop Music");
music.add(stop);
//Add Play Sound Effects as an option under Music
JMenuItem playEffect = new JMenuItem("Play Sound Effects");
music.add(playEffect);
//Add Mute Sound Effects as an option under Music
JMenuItem stopEffect = new JMenuItem("Mute Sound Effects");
music.add(stopEffect);
//Add About as an option under Help
JMenuItem about = new JMenuItem("About");
help.add(about);
//Add Controls as an option under Help
JMenuItem controls = new JMenuItem("Controls");
help.add(controls);
//Display the menu bar
container.setJMenuBar(mb);
//If Restart is selected, invoke this Action
restart.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        //Reset the message
        message = "The Daleks are invading!";
        //Reset the score
        score = 0;
        //Reset the level
        level = 1;
        //Wait for a key input
        getKey = true;
    }
});
//If Play is selected, invoke this Action
play.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        //If a song is not playing already, start playing it
        if (songPlaying == false) {
            playAudio();
        }
    }
});

```

```

        } else {
        }
    }
});
//If Stop is selected, invoke this Action
stop.addActionListener(new ActionListener() {
    //Stop playing the music
    public void actionPerformed(ActionEvent e) {
        stopAudio();
    }
});
//If Play Effect is selected, invoke this Action
playEffect.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        //If Sound Effects are turned off, turn them on
        if (soundEffect == false) {
            soundEffect = true;
        } else {
        }
    }
});
//If Stop Effects is selected, invoke this Action
stopEffect.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        //Mute all sound effects
        soundEffect = false;
    }
});
//If About is selected, invoke this Action
about.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        //Display a popup message box with the text
        JOptionPane.showMessageDialog(null, "The Daleks are invading London
once again!\n"
                                + "Earth needs your help once again, Doctor!\n");
    }
});
//If Controls is selected, invoke this Action
controls.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        //Display a popup message box with the text
        JOptionPane.showMessageDialog(null, "Controls: \n"
                                + "Press Space to fire!\n"
                                + "Move left and right with the arrow keys");
    }
});
//If Exit is selected, invoke this Action
exit.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        //Exit the game
        System.exit(1);
    }
});
//Gets the keyboard input handler and listens for input
addKeyListener(new KeyInputHandler());

```

```

    requestFocus();
    //Creates a basic 2 buffer strategy to load and display the images
    createBufferStrategy(2);
    strategy = getBufferStrategy();
    //Prepare for war
    initEntities();
}

private void startGame() {
    //Clear anything that may be left on the screen
    entities.clear();
    //Put the entities where they belong
    initEntities();
    //Set message to the text
    message = "The Daleks are invading";
    //If any buttons were being pressed, reset these values
    //to keep the TARDIS from moving or shooting at the start of the game
    leftPressed = false;
    rightPressed = false;
    firePressed = false;
    //If you are fighting Daleks, after each level, make them
    //drop faster for a harder level each time
    if ((level % 6) != 0) {
        for (int i = 0; i < entities.size(); i++) {
            Entity entity = (Entity) entities.get(i);
            entity.setHorizontalMovement(entity.setHorizontalMovement() * (1 +
level / 10));
        }
    }
    //If you beat the boss, you are rewarded with more shots in less time
    if ((level % 6) == 0) {
        firingInterval = firingInterval - 50;
    }
    //Begin the music
    playAudio();
}

private void initEntities() {
    //Our TARDIS, centered in the screen
    tardis = new TardisEntity(this, "tardis.png", 370, 530);
    //Display the TARDIS
    entities.add(tardis);
    //If you have reached a level of multiple 6,
    //it is time for a boss battle
    if ((level % 6) == 0) {
        for (int i = 0; i < bossLife; i++) {
            //Get the Dalek battle ship
            Entity boss = new BossEntity(this, "final.jpg", 250, 50);
            //Display the ship
            entities.add(boss);
        }
    } else {
        dalekCount = 0;
        for (int row = 0; row < 5; row++) {
            for (int x = 0; x < 12; x++) {

```

```

        //Get the Daleks
        Entity dalek = new DalekEntity(this, "dalek.png", 100 + (x * 50),
(50) + row * 50);
        //Display the 60 Daleks
        entities.add(dalek);
        //Increase the count of Daleks invading
        dalekCount++;
    }
}
}
}

```

```

public void gameLoop() {
    //Keeps track of the time of the game for movement and firing purposes
    long lastLoopTime = System.currentTimeMillis();

    while (gameRunning) {
        //Set delta to the current time - the last time the loop ran
        long delta = System.currentTimeMillis() - lastLoopTime;
        //Reset the last loop time to the current time
        lastLoopTime = System.currentTimeMillis();
        //Draw our graphics using our Buffer Strategy
        Graphics2D g = (Graphics2D) strategy.getDrawGraphics();
        g.setColor(Color.black);
        //Fill the background with a black color
        g.fillRect(0, 0, 800, 600);
        //Make the text white
        g.setColor(Color.white);
        g.drawString("Score: ", 0, 10);
        //Converts the score to a String to display to the Gamer
        Score = Integer.toString(score);
        g.drawString(Score, 40, 10);
        g.drawString("Level: ", 100, 10);
        //Converts the level to a String to display to the Gamer
        Level = Integer.toString(level);
        g.drawString(Level, 140, 10);

        if (!getKey) {
            //Move the enemies to the timer
            for (int i = 0; i < entities.size(); i++) {
                Entity entity = (Entity) entities.get(i);
                entity.move(delta);
            }
        }
        //Redraw the enemies after they move
        for (int i = 0; i < entities.size(); i++) {
            Entity entity = (Entity) entities.get(i);
            entity.draw(g);
        }
        //Check for collisions
        for (int p = 0; p < entities.size(); p++) {
            for (int s = p + 1; s < entities.size(); s++) {
                Entity entity1 = (Entity) entities.get(p);
                Entity entity2 = (Entity) entities.get(s);
                //If two entities take up the same place, they have collided
            }
        }
    }
}

```

```

        //Inform both entities (Dalek and Shot, Dalek and TARDIS) of the
collision
        if (entity1.collidesWith(entity2)) {
            entity1.collidedWith(entity2);
            entity2.collidedWith(entity1);
        }
    }
    //Remove anything in the Remove list from the game
    entities.removeAll(removeList);
    //Clear the list
    removeList.clear();
    //Update the game, drop the Daleks
    if (update) {
        for (int i = 0; i < entities.size(); i++) {
            Entity entity = (Entity) entities.get(i);
            entity.doLogic();
        }
        update = false;
    }
    //If a key input is needed
    if (getKey) {
        g.setColor(Color.white);
        //Display the message and wait to get a key input
        g.drawString(message, (700 -
g.getFontMetrics().stringWidth("message")) / 2, 25);
    }
    //Reset the game
    g.dispose();
    strategy.show();
    //Stop moving the TARDIS
    tardis.setHorizontalMovement(0);
    //If the <- arrow key is being pressed, move the TARDIS to the left
    if ((leftPressed) && (!rightPressed)) {
        tardis.setHorizontalMovement(-moveSpeed);
        //If the -> arrow key is being pressed, move the TARDIS to the right
    } else if ((rightPressed) && (!leftPressed)) {
        tardis.setHorizontalMovement(moveSpeed);
    }
    //If the space bar is pressed, ask the game to fire
    if (firePressed) {
        Fire();
    }

    try {
        Thread.sleep(10);
    } catch (Exception e) {
    }
}

//Update the game
public void updateGame() {
    update = true;
}

```

```

//If the Space bar is pressed
public void Fire() {
    //Check to see if enough time has passed since the last fired shot
    if (System.currentTimeMillis() - lastFire < firingInterval) {
        //If not enough time has passed, return and do not shoot
        return;
    }

    //If enough time has passed, set the last fired shot to the current time
    lastFire = System.currentTimeMillis();
    //Get the shot image and start it's trajectory from the top of the TARDIS
    ShotEntity shot = new ShotEntity(this, "shot.gif", tardis.getX() + 10,
tardis.getY() - 20);
    //Display the shot
    entities.add(shot);
    //If the Gamer wants sound effects on
    if(soundEffect)
        //Play the shot sound effect
        playShot();
}

//If a Dalek has been labeled to be destroyed
public void removeEntity(Entity entity) {
    //Add the Dalek to the list of things to be removed from the game
    removeList.add(entity);
}

//If the invasion is successful
public void notifyDeath() {
    //Message to alert the Gamer he has lost
    message = "You have been EXTERMINATED Try again.";
    //Reset the game
    level = 1;
    score = 0;
    //Wait for a key to be pressed
    getKey = true;
}

//If a Dalek has been destroyed
public void notifyDalekKilled() {
    //Remove the destroyed Dalek from the count of remaining Daleks
    dalekCount--;
    //Increase the score by 10
    score += 10;
    //If the Gamer wants sound effects on
    if(soundEffect)
        //Play the explosion sound effect
        playBlast();
    //If no Daleks remain, alert the Gamer he has won
    if (dalekCount == 0) {
        notifyWin();
    }
    //To make the level more challenging as the game progresses,
    //Make the remaining Daleks speed up

```

```

        for (int i = 0; i < entities.size(); i++) {
            Entity entity = (Entity) entities.get(i);

            if (entity instanceof DalekEntity) {
                //Increase the horizontal speed of the Daleks by 2%
                entity.setHorizontalMovement(entity.getHorizontalMovement() * 1.02);
            }
        }

//If the Dalek battle ship has been shot
public void notifyBossKilled() {
    //Decrease the life of the ship
    bossLife--;
    //If the Gamer wants sound effects on
    if(soundEffect)
        //Play the explosion sound effect
        playBlast();
    //If the boss has no life left
    if (bossLife == 0) {
        //Increase the score by 1000
        score += 1000;
        //Reset the boss for the next time he is faced
        bossLife = 35 + level;
        //Alert the gamer he won
        notifyWin();
    }
}

//If the Gamer has beaten the level
public void notifyWin() {
    //Change the message to tell the Gamer they won
    message = "Thank you Doctor!";
    //Increase the level
    level++;
    //Alert the game a Key must be pressed to continue
    getKey = true;
}

//Play the background music
public void playAudio() {
    try {
        //Get the background music file
        in = new FileInputStream(new File("./I_am_the_Doctor.wav"));
        //Convert the file into an AudioStream
        as = new AudioStream(in);
        //Play the background music
        AudioPlayer.player.start(as);
        //Inform the game that the music is playing
        songPlaying = true;
    } catch (FileNotFoundException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    } catch (IOException e) {
        // TODO Auto-generated catch block

```

```

        e.printStackTrace();
    }
}

//Play the shot sound effect
public void playShot() {
    try {
        //Get the shot file
        in = new FileInputStream(new File("./laser.WAV"));
        //Convert the file into an AudioStream
        as = new AudioStream(in);
        //Play the sound effect
        AudioPlayer.player.start(as);
    } catch (FileNotFoundException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    } catch (IOException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
}

//Play the explosion blast sound effect
public void playBlast() {
    try {
        //Get the explosion blast file
        in = new FileInputStream(new File("./blast1.wav"));
        //Convert the file into an AudioStream
        as = new AudioStream(in);
        //Play the sound effect
        AudioPlayer.player.start(as);
    } catch (FileNotFoundException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    } catch (IOException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
}

//Play the EXTERMINATE sound effect
public void playDeath() {
    //If the Gamer has not turned off the sound effects
    if(soundEffect){
        try {
            //Get the exterminate file
            in = new FileInputStream(new File("./exterminate.wav"));
            //Convert the file into an AudioStream
            as = new AudioStream(in);
            //Play the sound effect
            AudioPlayer.player.start(as);
        } catch (FileNotFoundException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        } catch (IOException e) {

```

```

        // TODO Auto-generated catch block
        e.printStackTrace();
    }
}
//If the Gamer has the sound effects muted, do nothing
else{}
}

//If the Gamer requests the music be turned off, this class is called
public void stopAudio() {
    //Stop playing the music
    AudioPlayer.player.stop(as);
    //Alert the game that the music is no longer playing
    songPlaying = false;
}

//Retrieves Keyboard Input
private class KeyInputHandler extends KeyAdapter {

    private int pressCount = 1;

    public void keyPressed(KeyEvent e) {
        //Alerts the game that a key has been pressed
        if (getKey) {
            return;
        }
        //If the <- arrow key is pressed, move left
        if (e.getKeyCode() == KeyEvent.VK_LEFT) {
            leftPressed = true;
        }
        //If the -> arrow key is pressed, move right
        if (e.getKeyCode() == KeyEvent.VK_RIGHT) {
            rightPressed = true;
        }
        //If the space bar is pressed, fire
        if (e.getKeyCode() == KeyEvent.VK_SPACE) {
            firePressed = true;
        }
    }

    public void keyReleased(KeyEvent e) {
        //Alerts the game that a key has been released
        if (getKey) {
            return;
        }
        //If the <- arrow key is released, stop moving
        if (e.getKeyCode() == KeyEvent.VK_LEFT) {
            leftPressed = false;
        }
        //If the -> arrow key is released, stop moving
        if (e.getKeyCode() == KeyEvent.VK_RIGHT) {
            rightPressed = false;
        }
        //If the space bar is released, stop firing
        if (e.getKeyCode() == KeyEvent.VK_SPACE) {

```

```

        firePressed = false;
    }
}

public void keyTyped(KeyEvent e) {
    //Keeps track of whether or not a key has been pressed
    if (getKey) {
        if (pressCount == 1) {
            //If a key has been pressed, alert the game that in no longer
            //has to wait for keyboard input to continue
            getKey = false;
            //Start the game
            startGame();
            pressCount = 0;
        } else {
            pressCount++;
        }
    }
}

}

public static void main(String argv[]) {
    Game g = new Game();
    g.gameLoop();
}
}

```

Image Loader

```
import java.awt.GraphicsConfiguration;
import java.awt.GraphicsEnvironment;
import java.awt.Image;
import java.awt.Transparency;
import java.awt.image.BufferedImage;
import java.io.IOException;
import java.net.URL;
import java.util.HashMap;

import javax.imageio.ImageIO;

public class LoadImages {
    private static LoadImages image = new LoadImages();
    //Load the Images
    public static LoadImages get() {
        return image;
    }

    //Cached Sprite map to increase the speed we receive the images
    private HashMap<String, Sprite> images = new HashMap<String, Sprite>();

    //Get the images from the cache
    public Sprite getSprite(String ref) {
        //If the image is already cached, use that image
        if (images.get(ref) != null) {
            return (Sprite) images.get(ref);
        }

        //If the image is not in the cache, go get the image from Class Loader
        BufferedImage sourceImage = null;

        try {
            //Retrieves the image resource
            URL url = this.getClass().getClassLoader().getResource(ref);
            //If the image cannot be found, print out the error
            if (url == null) {
                System.err.println("Can't find ref: " + ref);
                System.exit(0);
            }

            //Use ImageIO to read the image in
            sourceImage = ImageIO.read(url);
        } catch (IOException e) {
            System.err.println("Failed to load: " + ref);
            System.exit(0);
        }

        //Create the image
        GraphicsConfiguration gc =
GraphicsEnvironment.getLocalGraphicsEnvironment().getDefaultScreenDevice().getDefault
Configuration();
```

```
        Image image = gc.createCompatibleImage(sourceImage.getWidth(),
sourceImage.getHeight(), Transparency.BITMASK);

        //Draw the image
        image.getGraphics().drawImage(sourceImage, 0, 0, null);

        //Cache the image for future use to avoid buffer issues
        Sprite sprite = new Sprite(image);
        images.put(ref, sprite);

        return sprite;
    }
}
```

Sprite

```
import java.awt.Graphics;
import java.awt.Image;

public class Sprite {
    private Image image;
    //Create the sprite of the image requested
    public Sprite(Image image) {
        this.image = image;
    }

    //Get and Return the width of the image. This will be used to
    //detect collisions and draw the image
    public int getWidth() {
        return image.getWidth(null);
    }

    //Get and Return the height of the image. This will be used to
    //draw and space images properly
    public int getHeight() {
        return image.getHeight(null);
    }

    //Draw the desired image using Graphics
    public void draw(Graphics g, int x, int y) {
        g.drawImage(image, x, y, null);
    }
}
```

Entity

```
import java.awt.Graphics;
import java.awt.Rectangle;

public abstract class Entity {
    protected double x;
    protected double y;
    protected Sprite sprite;
    protected double dx;
    protected double dy;
    private Rectangle entity1 = new Rectangle();
    private Rectangle entity2 = new Rectangle();

    //Load the Image of the Dalek, TARDIS, boss, or shot from the cache
    public Entity(String ref, int x, int y) {
        this.sprite = LoadImages.get().getSprite(ref);
        this.x = x;
        this.y = y;
    }

    //Turns X into a integer for easier manipulation
    public int getX() {
        return (int) x;
    }

    //Turns Y into a integer for easier manipulation
    public int getY() {
        return (int) y;
    }

    //Move the entity based on the speed (delta)
    public void move(long delta) {
        x += (delta * dx) / 1000;
        y += (delta * dy) / 1000;
    }

    //Set the horizontal speed of the entity
    public void setHorizontalMovement(double dx) {
        this.dx = dx;
    }

    //Sets the vertical speed of the shot moving up and the Daleks dropping
    public void setVerticalMovement(double dy) {
        this.dy = dy;
    }

    //Get the horizontal speed of the entity
    public double getHorizontalMovement() {
        return dx;
    }

    //Get the horizontal speed of the entity
    public double getVerticalMovement() {
```

```

        return dy;
    }

    //Draw the entity
    public void draw(Graphics g) {
        sprite.draw(g, (int) x, (int) y);
    }

    //If logic is requested, it is done
    public void doLogic() {
    }

    //Checks for collisions with entities
    public boolean collidesWith(Entity other) {
        entity1.setBounds((int) x, (int) y, sprite.getWidth(), sprite.getHeight());
        entity2.setBounds((int) other.x, (int) other.y, other.sprite.getWidth(),
other.sprite.getHeight());

        return entity1.intersects(entity2);
    }

    //Collision detection done within each Entity class
    public abstract void collidedWith(Entity other);
}

```

TARDIS Entity

```
public class TardisEntity extends Entity {
    private Game game;
    //Creates the TARDIS at the desired starting point of X, Y
    public TardisEntity(Game game, String ref, int x, int y) {
        super(ref, x, y);
        this.game = game;
    }

    //Moves the TARDIS when a key is pressed
    public void move(long delta) {
        //If we have reached the edge of the screen on the left, do not move
        if((dx < 0) && (x < 10)) {
            return;
        }
        //If we have reached the edge of the screen on the right, do not move
        if((dx > 0) && (x > 750)) {
            return;
        }
        super.move(delta);
    }

    //If a collision has occurred, this function is called.
    public void collidedWith(Entity other) {
        //If the TARDIS has hit a Dalek, stop the music and end the game
        if(other instanceof DalekEntity) {
            game.stopAudio();
            game.playDeath();
            game.notifyDeath();
        }
        //If the TARDIS has hit the boss, stop the music and end the game
        if(other instanceof BossEntity) {
            game.stopAudio();
            game.playDeath();
            game.notifyDeath();
        }
    }
}
```

ShotEntity

```
public class ShotEntity extends Entity {

    //Speed at which the shot will move, negative because
    //the shot must move up toward the Daleks
    private double moveSpeed = -300;
    private Game game;
    //Keeps track of if a shot has been used (true) or not (false)
    private boolean used = false;

    //Create the shot at the spot the Tardis is currently at
    public ShotEntity(Game game, String sprite, int x, int y) {
        super(sprite, x, y);
        this.game = game;
        dy = moveSpeed;
    }

    //Moves the shot based on the speed of the game
    public void move(long delta) {
        super.move(delta);
        //If the shot has gone off the screen, remove itself from the game
        if(y < -100) {
            game.removeEntity(this);
        }
    }

    //Collision detection
    public void collidedWith(Entity other) {
        //Prevents a shot from hitting a Dalek twice.
        //If true, do not collide.
        if(used) {
            return;
        }

        //If the shot collided with a Dalek, remove the shot and remove the Dalek
        if(other instanceof DalekEntity) {
            // remove the affected entities
            game.removeEntity(this);
            game.removeEntity(other);
            //Allows the game to update the count of Daleks and score
            game.notifyDalekKilled();
            used = true;
        }

        //If the shot has hit the boss, remove the shot and the boss
        if(other instanceof BossEntity) {
            game.removeEntity(this);
            game.removeEntity(other);
            //Allows the game to update the score
            game.notifyBossKilled();
            used = true;
        }
    }
}
```

Dalek Entity

```
public class DalekEntity extends Entity {
    //Speed Daleks will move
    private double moveSpeed = 75;
    private Game game;

    //Creates Dalek with set speed
    public DalekEntity(Game game, String ref, int x, int y) {
        super(ref, x, y);

        this.game = game;
        dx = moveSpeed;
    }

    public void move(long delta) {
        //If the Daleks reach the end of the screen on the left,
        //drop down the screen and reverse direction
        if ((dx < 0) && (x < 10)) {
            game.updateGame();
        }
        //If the Daleks reach the end of the screen on the right,
        //drop down the screen and reverse direction
        if ((dx > 0) && (x > 750)) {
            game.updateGame();
        }

        super.move(delta);
    }

    public void doLogic() {
        //Reverse direction
        dx = -dx;
        //Drop down on the screen
        y += 10;
        //If the Daleks reach the bottom of the Frame and have not yet touched the
TARDIS,
        //notify the game that the Daleks have invaded
        if (y > 540) {
            game.notifyDeath();
        }
    }

    //Collision detection
    public void collidedWith(Entity other) {}
}
```

Dalek Battle Ship Boss Entity

```
public class BossEntity extends Entity {
    //Speed the boss ship will move
    private double moveSpeed = 100;
    private Game game;

    //Create the Dalek boss ship
    public BossEntity(Game game, String ref, int x, int y) {
        super(ref, x, y);
        this.game = game;
        dx = moveSpeed;
    }

    //Move the boss ship
    public void move(long delta) {
        //If the Boss ship reaches the end of the screen on the left,
        //drop down the screen and reverse direction
        if ((dx < 0) && (x < 10)) {
            game.updateGame();
        }
        //If the Boss ship reaches the end of the screen on the right,
        //drop down the screen and reverse direction
        if ((dx > 0) && (x > 520)) {
            game.updateGame();
        }
        super.move(delta);
    }

    public void doLogic() {
        //Reverse direction
        dx = -dx;
        //Drop down on the screen
        y += 15;

        //If the Dalek ship reach the bottom of the Frame and has not yet touched the
        TARDIS,
        //notify the game that the Daleks have invaded
        if (y > 540) {
            game.notifyDeath();
        }
    }

    //Collision detection
    public void collidedWith(Entity other) {}
}
```

Dalek Invaders

Brian Cowan

Overview

- Purpose
- Programming Environment
- TimeLine
- What is Next?
- Challenges Overcome
- Demonstration

Purpose

- I wanted to make a fun game and take it to another level.
- Doctor Who premiered on November 23, 1963, 50 years ago tomorrow.
- A retro looking game for a retro television show

Programming Environment

- Eclipse
- Java
- JPanel/JFrame

Timeline

- Summer 2012 – Came up with the idea and design layout with rules
- Fall 2012 – Began project
 - August – Did research into how to display the project. Decided on Canvas
 - August - October – Got layout to print
 - October - November – Got Actors and Enemies to print correctly

TimeLine

- Winter 2012-2013 – Movement
 - December – January - Actor was able to be controlled by arrows
 - January – February – Researched how the timer works in Java and how to use it properly
 - February – April – Enemies began to move to a timer
- Summer 2013 – Collision Detection
 - April – July – Debated how to shoot enemies. Followed timer
 - July – August – Collision detection with ship and enemies and enemies with borders of canvas

Timeline

- Fall 2013
 - August – October – Changing the speed enemies moved
 - October – November – Debugging, Added Music and Boss battles
 - Early November – Virus found
 - November 7-16 – Rebuilding the project

What is next?

- New levels
- New designs
- New enemies

Challenges

- Keyboard input
- Determining the correct window size
- Movement
- Timers
- Collision Detection of fired shot, enemies touching your ship and reaching Earth
- Text
- Numbers
- Music
- Boss
- Buffering Images

Questions?

The Daleks are invading!



Space Invaders: Doctor WHO edition

File Music Help

Restart

Exit

Level: 1

The Daleks are invading!



Space Invaders: Doctor WHO edition

File

Music

Help

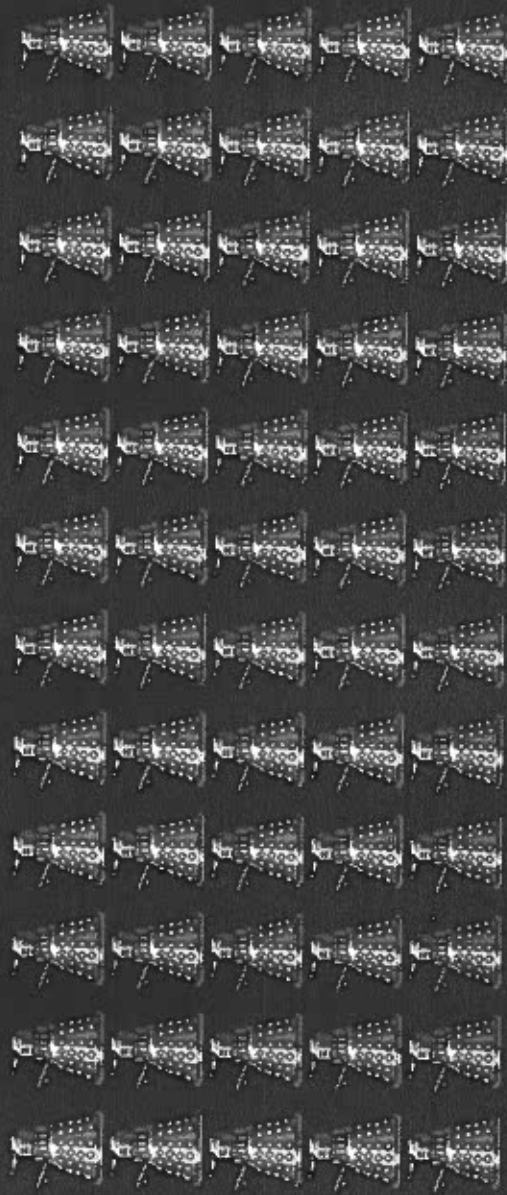
Score

Level: 1

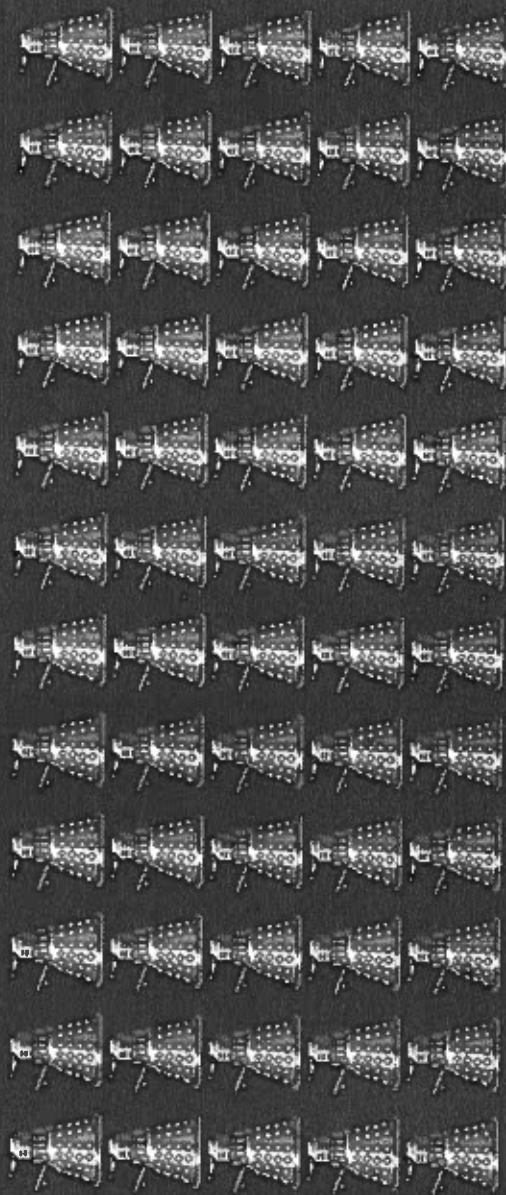
Play Music

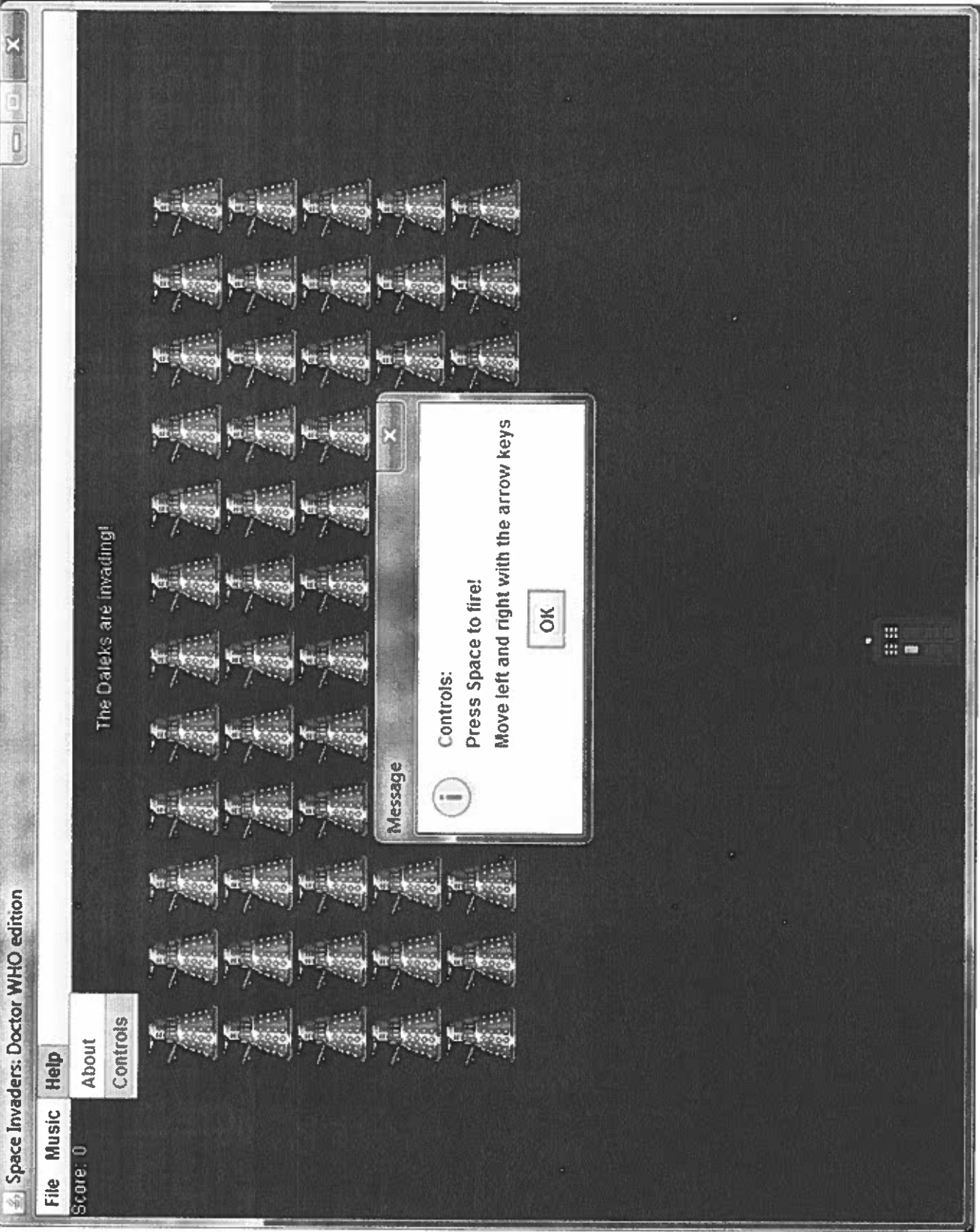
Stop Music

The Daleks are invading!



The Daleks are invading!





Space Invaders: Doctor WHO edition

File Music Help

Score: 0

About

Controls

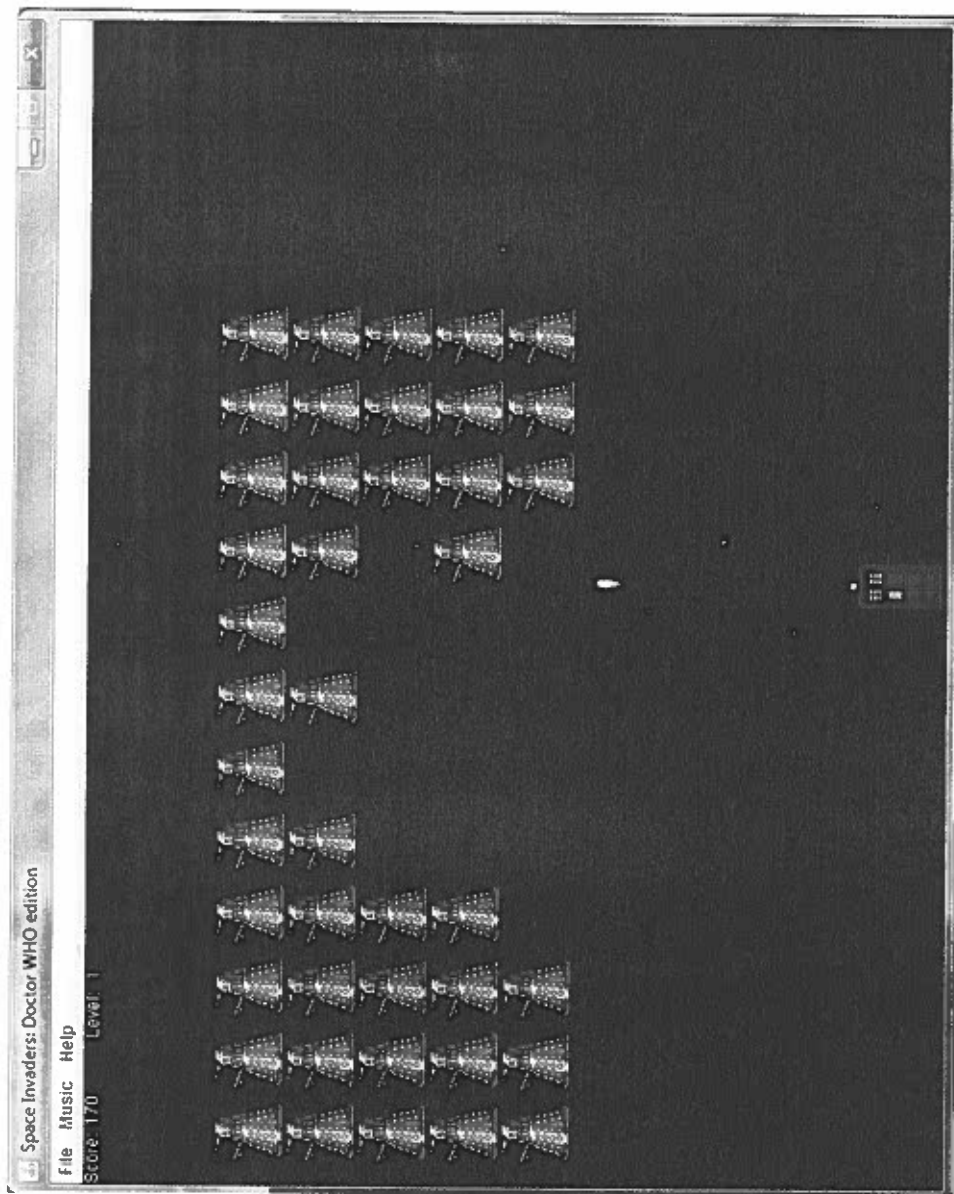
The Daleks are invading!

Message



Controls:
Press Space to fire!
Move left and right with the arrow keys

OK



Space Invaders: Doctor WHO edition

File Music Help

Score: 600 Level: 2

You stopped them! Oh no! More are coming! Help Doctor!





Space Invaders: Doctor WHO edition

File Music Help

Score: 0 Level: 1

